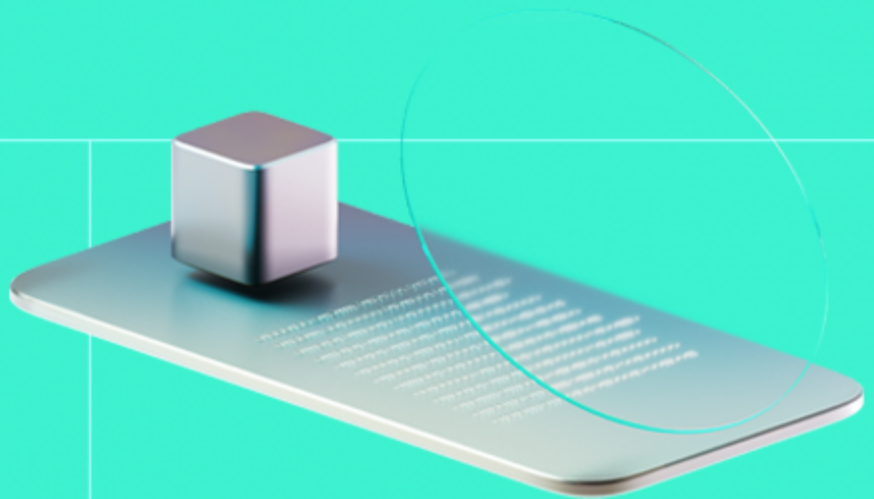




Smart Contract Code Review And Security Analysis Report

Customer: VEREM

Date: 05/03/2026



We express our gratitude to the VEREM team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

The VEREM ecosystem integrates certified physical emeralds into blockchain technology through a structure that prioritizes transparency, governance, sustainability, and systemic risk mitigation.

Document

Name	Smart Contract Code Review and Security Analysis Report for VEREM
Audited By	Khrystyna Tkachuk
Approved By	Ivan Bondar
Website	https://verem.org/
Changelog	03/03/2026 - Preliminary Report
	05/03/2026 - Final Report
Platform	BSC
Language	Solidity
Tags	ERC20
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/veremrwa/VeremContracts
Commit	b6bff64
Contract Address	0x862829e8dCf0Fd939D61d1b0D4a3aaf983ee4F73

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

2	0	2	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	0

Vulnerability	Severity	Status
F-2026-15271 - Redundant Override of decimals() Function	Info	Accepted
F-2026-15272 - Lack of NatSpec Documentation	Info	Accepted



Documentation quality

- Functional requirements are missed.
- Technical description is not provided.

Code quality

- The development environment is not configured.

Test coverage

Code coverage of the project is **0%**.

- Test cases are not provided.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	11
Appendix 1. Definitions	12
Severities	12
Potential Risks	12
Appendix 2. Scope	13
Appendix 3. Additional Valuables	14

System Overview

VEREM — is an ERC-20 token that mints the entire initial supply to the deployer. No additional minting is allowed. The token includes a transfer-enable mechanism that is disabled at deployment and can be switched on only once by the owner to enable transfers.

It has the following attributes:

- Name: Verified Emeralds
- Symbol: VEREM
- Decimals: 18
- Total supply: 50 000 000.

Privileged roles

- The owner of the VEREM contract can add accounts to the exclusion list (`excludeFromEnable()`), allowing them to transfer tokens even when trading is disabled. The owner can call `enableTrading()` only once, after which transfers will be enabled for all users. The owner is excluded from the transfer restrictions. In the current deployed contract on BSC chain the owner is renounced.

Potential Risks

- **Centralized Minting to a Single Address:** The project concentrates minting tokens in a single address, raising the risk of fund mismanagement or theft, especially if key storage security is compromised.
- **Control Risks from Whitelisted Users:** Granting significant privileges to whitelisted addresses (`isExcludedFromEnable`) without adequate safeguards may introduce operational uncertainty and undermine user confidence. Whitelisted accounts are able to transfer tokens while trading is disabled, enabling privileged pre-launch distribution and private OTC-like transfers. This mechanism can be misused for stealth allocations, early liquidity provisioning, or selective dumping once trading is enabled, creating potential fairness and market integrity concerns.
- **Single Points of Failure and Control:** Part of the system rely on a limited set of privileged roles and operational processes. While this setup simplifies coordination, it also means that overall availability and decision execution may depend on a small number of entities, increasing sensitivity to operational issues or targeted compromise. However, in the current deployed contract on the BSC chain, the owner is renounced.

Findings

Vulnerability Details

[F-2026-15271](#) - Redundant Override of decimals() Function - Info

Description: The **VEREM** contract inherits from OpenZeppelin's **ERC20** implementation, which already provides a functional `decimals()` method returning `18` by default.

In the **VEREM** contract, the function is overridden as follows:

```
function decimals() public view virtual override returns (uint8) {
    return 18;
}
```

This override does not introduce any new logic or customization. Since OpenZeppelin's **ERC20** implementation already returns `18` decimals by default, the override is functionally redundant. Given that, it unnecessarily increases contract bytecode size, slightly impacts deployment cost, and reduces overall code clarity without providing any functional benefit.

Assets:

- Verem [<https://github.com/veremrwa/VeremContracts>]

Status: Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Consider removing the redundant `decimals()` override and rely on the inherited OpenZeppelin **ERC20** implementation.

Resolution:

Accepted, the client is aware and accepting the finding.

F-2026-15272 - Lack of NatSpec Documentation - Info

Description:

The **VEREM** contract implements an ERC20 token with a custom trading enablement mechanism. However, the contract source code does not include any NatSpec documentation.

The absence of NatSpec comments represents a deviation from best practices, reducing the clarity, readability, and usability of the contract. Comprehensive documentation is essential for clearly communicating the purpose, functionality, and intended usage of contracts and their functions.

Without proper NatSpec annotations, integrators, auditors, and automated documentation tools may face difficulties in understanding expected behaviors, assumptions, and parameter constraints.

Assets:

- Verem [<https://github.com/veremrwa/VeremContracts>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to add comprehensive NatSpec documentation to all public and external functions, including relevant tags such as `@notice`, `@dev`, `@param`, and `@return`. This will improve clarity, usability, maintainability, and compatibility with automated documentation and analysis tools.

Resolution:

Accepted, the client is aware and accepting the finding.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/veremrwa/VeremContracts
Commit	b6bff64083b57fcd6677467f3d020a33b9cdd321
Contract Address	0x862829e8dCf0Fd939D61d1b0D4a3aaf983ee4F73
Whitepaper	N/A
Requirements	
Technical Requirements	

Asset	Type
Verem [https://github.com/veremrwa/VeremContracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.